

Prompt: Eksperymenty z głowicą density dla lokalizacji przyrostów otolitu (DINO + CORAL + Density)

Kontekst projektu

Pracujesz na istniejącym projekcie do szacowania wieku otolitów śledzia. Architektura opiera się na backbone DINO (Vision Transformer szkolony samodzielnie, self-supervised) z dwiema głowicami:

- **Głowica CORAL** — regresja porządkowa wieku otolitu (CORAL: COnsistent RANk Logits, zapewnia monotoniczność rangi)
- **Głowica density** — ma za zadanie wskazać na obrazie miejsca, gdzie znajdują się przyrosty (growth increments / annuli), bez ręcznego etykietowania tych lokalizacji

Celem jest przetestowanie różnych wariantów konfiguracji i trenowania głowicy density, aby sprawdzić, które podejście daje najlepsze wyniki lokalizacji przyrostów przy użyciu słabonadzorowanego uczenia (weakly supervised learning).

Krok 0: Zbadanie istniejącego projektu

Zanim zaczniesz implementować nowe warianty, wykonaj następujące kroki:

1. Zidentyfikuj strukturę projektu:

- Znajdź główny plik treningowy (np. `train.py`, `main.py`, `run.py`)
- Znajdź definicję modelu (np. `model.py`, `models/`, `network.py`)
- Znajdź plik konfiguracyjny (np. `config.yaml`, `config.json`, `args.py`)
- Znajdź dataset / dataloader (np. `dataset.py`, `data/`)
- Znajdź funkcje straty (np. `losses.py`, w kodzie modelu)
- Znajdź logowanie (TensorBoard, W&B, CSV logs)
- Znajdź checkpoint'y i zapis modelu

2. Zidentyfikuj obecne komponenty:

- Jaki backbone DINO jest używany? (DINO ViT-S/16, DINO ViT-B/8, DINOv2?)
- Jak jest skonfigurowana głowica CORAL? (liczba klas, loss)
- Czy głowica density już istnieje? Jeśli tak, jak jest zaimplementowana?
- Jaki jest format danych wejściowych? (rozmiar obrazu, normalizacja, liczba kanałów)
- Jaki jest format etykiet wieku? (liczba całkowita, zakres)
- Jakie są metryki ewaluacji?

3. Wypisz na ekran podsumowanie:

- Nazwy plików i kluczy klas
- Kształty tensorów na każdym etapie (batch, channels, H, W)
- Obecne loss functions
- Obecny training loop (kroki na epokę, optimizer, scheduler)

Architektura bazowa (wspólna dla wszystkich wariantów)

Backbone: DINO (zamrożony)

```
Backbone: DINO ViT-S/16 (lub DINOv2 ViT-S/14) – FROZEN
Input:    [B, 3, 224, 224] (lub rozmiar z projektu)
Output:   [B, N_patches, D] – cechy patchy
          [B, D]           – token [CLS]
```

```
N_patches = (H/patch_size) * (W/patch_size)
D = 384 (ViT-S) lub 768 (ViT-B)
```

Głowica CORAL (wiek otolitu)

```
Input: [B, D] (token [CLS])
Output: [B, K-1] (K = maksymalny wiek + 1, binarne klasyfikatory porządkowe)
Loss:   CORAL ordinal loss (binarna entropia krzyżowa na każdym z K-1 klasyfikatorów)
```

Referencja: Cao, Mirjalili, Raschka (2020)
URL: <https://arxiv.org/abs/1901.07884>
GitHub: <https://github.com/Raschka-research-group/coral-pytorch>

Głowica density (lokalizacja przyrostów) – WARIANTY

Główny przedmiot eksperymentów. Poniżej opisano 7 wariantów.

Wariant 1: Baseline – Density integral constraint

Literatura

- Cholakal et al., "Object Counting and Instance Segmentation with Image-Level Supervision", CVPR 2019
- URL: https://openaccess.thecvf.com/content_CVPR_2019/papers/Cholakal_Object_Counting_and_Instance_Segmentation_With_Image-Level_Supervision_CVPR_2019_paper.pdf

Opis

Najprostszy wariant: mapa gęstości musi sumować się do wieku. Integral mapy gęstości = wiek z CORAL (lub etykieta wieku). Brak innych sygnałów lokalizujących.

Architektura

```
Input: [B, D, N_patches] (cechy patchy z DINO)
Head:  Linear(D, 1) → ReLU → reshape do [B, H_patches, W_patches]
Output: [B, H_patches, W_patches] – mapa gęstości (nieujemna)
```

Loss function

```
# Density integral loss
density_map = model.density_head(patch_features) # [B, N_patches]
density_map = density_map.view(B, H_patches, W_patches)
predicted_count = density_map.sum(dim=(1, 2)) # [B] – integral mapy

# Etykieta wieku (z CORAL lub z ground truth)
age_labels = batch["age"] # [B] – wiek jako liczba całkowita

loss_density = F.mse_loss(predicted_count, age_labels.float())

# Alternatywnie: loss z głowicy CORAL jako soft target
coral_output = model.coral_head(cls_token) # [B, K-1]
predicted_age = coral_output.sum(dim=1) # soft count
loss_density_coral = F.mse_loss(predicted_count, predicted_age.detach())

# łączny loss
loss = loss_coral + lambda_density * loss_density
```

Parametry

- `lambda_density`: 1.0 (start), do strojenia w zakresie [0.1, 0.5, 1.0, 2.0]
- Optimizer: Adam, lr=1e-4 (głowica density), backbone zamrożony

Metryki

- Age MAE (głowica CORAL)
- Błąd sumy density: $|\text{predicted_count} - \text{true_age}| / \text{true_age}$
- Sparsity mapy: ułamek patchy z wartością > 0 (powinien być umiarkowany)
- Entropia mapy (powinna być umiarkowana – nie za niska = jeden punkt, nie za wysoka = równomierna)

Wariant 2: Cholakkal — Image-level supervision z classification + density heads

Literatura

- Cholakkal et al., "Object Counting and Instance Segmentation with Image-Level Supervision", CVPR 2019
- URL: https://openaccess.thecvf.com/content_CVPR_2019/papers/Cholakkal_Object_Counting_and_Instance_Segmentation_With_Image-Level_Supervision_CVPR_2019_paper.pdf

Opis

Główica klasyfikacji (obecność/brak kategorii) generuje mapy kategorii, z których ekstrahowane są lokalne maksima jako pseudo-lokalizacje. Główica density uczy się na tych pseudo-lokalizacjach. W naszym przypadku: zamiast kategorii obiektów, "kategoria" to "przyrost" — model uczy się, gdzie są przyrosty, mając tylko wiek.

Architektura

```
Backbone: DINO ViT-S/16 (zamrożony)
Główica A (classification/attention):
    Linear(D, 1) → Sigmoid → [B, N_patches] — mapa uwagi dla przyrostów
Główica B (density):
    Linear(D, 1) → ReLU → [B, N_patches] → reshape → [B, H, W] — mapa gęstości

# Reshape z patch tokens do mapy 2D
# patch_tokens: [B, N_patches, D] → [B, D, H_patches, W_patches] → głowice
```

Loss function

```
# 1. Classification branch — uczy się mapy uwagi
class_map = model.class_head(patch_features) # [B, N_patches]
class_map = class_map.view(B, H_patches, W_patches)
# Pseudo-ground-truth: lokalne maksima na mapie klasyfikacji
# Liczba lokalnych maksimów = wiek (w zakresie subitizing 1-4)
# Powyżej subitizing: ranking loss (predicted >= 5)

# Dla obrazów z wiekiem <= 4:
local_maxima = find_local_maxima(class_map, radius=1) # [B, N_patches] binary
# Density loss w punktach maksimów
loss_density_points = (density_map * local_maxima).sum(dim=(1,2))
# Sprawdź, czy liczba maksimów odpowiada wiekowi
predicted_count_from_maxima = local_maxima.sum(dim=1)
loss_count = F.mse_loss(predicted_count_from_maxima.float(), age_labels.float())

# Dla obrazów z wiekiem > 4:
# Ranking loss — kara za under-counting
loss_ranking = F.relu(5.0 - predicted_count_from_maxima.float()).mean()

# 2. Density branch — integral = count
predicted_count = density_map.sum(dim=(1, 2))
```

```

loss_density_integral = F.mse_loss(predicted_count, age_labels.float())

# 3. Global count loss
loss_global = F.mse_loss(predicted_count, age_labels.float())

loss = loss_coral + lambda_class * loss_classification + lambda_density * (loss_dens:

```

Parametry

- `lambda_class`: 1.0
- `lambda_density`: 1.0
- `lambda_rank`: 0.1
- Radius dla local maxima: `r=1` (stride 1)
- Subitizing range: 1-4 (dla małych wieków; powyżej — ranking loss)

Kluczowa różnica względem Wariantu 1

Dodatkowa głowica klasyfikacji generuje pseudo-lokalizacje, które douczają głowicę density — model uczy się nie tylko sumy, ale też rozmieszczenia.

Wariant 3: MATT — Multi-branch auxiliary density with consistency loss

Literatura

- Liu et al., "Towards Using Count-level Weak Supervision for Crowd Counting", 2020
- URL: <https://arxiv.org/pdf/2003.00164v1>

Opis

Główna głowica density + 4 głowice pomocnicze. Głowice pomocnicze przewidują mapy gęstości o różnych rozmiarach jądra Gaussa. Consistency loss wymusza, aby wszystkie mapy zgadzały się z główną. Trenowane asymetrycznie: gradienty z głowic pomocniczych nie płyną do głównej.

Architektura

Backbone: DINO ViT-S/16 (zamrożony)
Patch features: [B, N_patches, D]

Główne głowice:

CORAL head: Linear(D, K-1) na [CLS] token

Primary density head: Linear(D, 128) → ReLU → Linear(128, 64) → ReLU → Linear(64, 1)
Output: [B, N_patches] → reshape → [B, H_patches, W_patches]

Głowice pomocnicze (4 sztuki, porzucone po treningu):

Aux head 1: Linear(D, 128) → ReLU → Linear(128, 64) → ReLU → Linear(64, 1) → ReLU

Aux head 2: (ta sama architektura)

```
Aux head 3: (ta sama architektura)
Aux head 4: (ta sama architektura)
```

Każda głowica pomocnicza przewiduje mapę gęstości, która jest konwolwowana różnymi jądrami

```
Kernel 1: 3×3,  $\sigma=1$ 
Kernel 2: 5×5,  $\sigma=1$ 
Kernel 3: 3×5,  $\sigma=1$ 
Kernel 4: 5×3,  $\sigma=1$ 
(wartości znormalizowane tak, że integral = 1)
```

Loss function

```
# 1. CORAL loss (głowica wieku)
loss_coral = coral_loss(coral_output, age_labels)

# 2. Primary density – integral = wiek
primary_density = model.primary_density_head(patch_features) # [B, N_patches]
primary_density = primary_density.view(B, H_patches, W_patches)
primary_count = primary_density.sum(dim=(1, 2))
loss_primary_count = F.mse_loss(primary_count, age_labels.float())

# 3. Auxiliary density – integral = wiek (gradienty NIE płyną do primary)
for i, aux_head in enumerate(model.aux_heads):
    aux_density = aux_head(patch_features) # [B, N_patches]
    aux_density = aux_density.view(B, H_patches, W_patches)
    aux_count = aux_density.sum(dim=(1, 2))
    loss_aux_count = F.mse_loss(aux_count, age_labels.float())

# 4. Consistency loss: skonwolwowana mapa aux powinna pasować do primary
# (primary traktowane jako FIXED – detach)
kernel = gaussian_kernels[i] # [3,3], [5,5], [3,5], [5,3]
aux_smoothed = F.conv2d(aux_density.unsqueeze(1), kernel.unsqueeze(0).unsqueeze(0))
loss_consistency = F.mse_loss(aux_smoothed.squeeze(1), primary_density.detach())

loss_aux = loss_aux_count + loss_consistency

loss = loss_coral + lambda_primary * loss_primary_count + lambda_aux * sum(loss_aux)
```

Parametry

- `lambda_primary`: 1.0
- `lambda_aux`: 1.0
- Liczba głowic pomocniczych: 4 (najlepsza według paperu)
- Asymetria: gradienty z aux NIE płyną do primary (`.detach()` na primary w consistency loss)
- Po treningu: zachowaj tylko primary head i CORAL head

Kluczowa różnica

Główce pomocnicze z różnymi jądrami Gaussa wymuszają, aby mapa gęstości była odporna na różne skale przyrostów. Consistency loss zapobiega degeneracji mapy do jednego punktu.

Wariant 4: Count2Density — Historical Map Bank + contrastive spatial regularizer

Literatura

- "Count2Density: Crowd Density Estimation without Location-level Annotations", 2025
- URL: <https://arxiv.org/pdf/2509.03170>

Opis

Zamiast bezpośredniego nadzoru punktowego, model utrzymuje "Historyczny Bank Map" (Historical Map Bank) — eksponencjalną średnią ruchomą (EMA) przewidywanych map gęstości dla każdego obrazu treningowego. Mapy te służą jako pseudo-etykiety (pseudo-density maps). Dodatkowo, kontrastywny regularizator przestrzenny zachęca do podobnych reprezentacji w gęstych regionach i różnych w tle.

Architektura

```
Backbone: DINO ViT-S/16 (zamrożony)
Feature extractor:  $\Psi$  = DINO (zamrożony)
Density decoder:  $\Phi$  = Linear(D, 256)  $\rightarrow$  ReLU  $\rightarrow$  Linear(256, 128)  $\rightarrow$  ReLU  $\rightarrow$  Linear(128, 1)
Input: [B, N_patches, D]
Output: [B, N_patches]  $\rightarrow$  reshape  $\rightarrow$  [B, H_patches, W_patches]
```

Historical Map Bank:

- Słownik: {image_id: map_tensor} — inicjalizowany BASNet (unsupervised saliency)
- Aktualizowany co epokę: $H_i = \alpha * \hat{M}_i + (1-\alpha) * H_i$, $\alpha=0.7$
- Mapa znormalizowana przez max, wygładzona filtrem Gaussa
- y_i lokalizacji próbkowanych bez zwracania z rozkładu hipergeometrycznego

Contrastive Spatial Regularizer:

- Threshold na przewidywanej mapie $\rightarrow$ crowded regions vs background
- Positive pairs: cechy z crowded regions tego samego obrazu
- Negative pairs: crowded vs background
- Temperatura: $\tau=0.07$

Loss function

```
# 1. CORAL loss
loss_coral = coral_loss(coral_output, age_labels)

# 2. Map loss — pseudo-density map z Historical Map Bank
predicted_map = model.density_decoder(patch_features) # [B, H, W]

for i in range(B):
```

```

img_id = batch["image_id"][i]
if img_id in historical_map_bank:
    # Pobierz pseudo-mapę z banku
    pseudo_map = historical_map_bank[img_id] # [H, W]
    # Normalizuj przez max, wygładź Gaussa
    pseudo_map = gaussian_smooth(pseudo_map / (pseudo_map.max() + 1e-8))
    # Próbuj y_i lokalizacji (y_i = age_labels[i])
    sampled_points = sample_locations(pseudo_map, n=age_labels[i].item())
    # Stwórz pseudo-density map z próbkowanych punktów
    pseudo_density = make_density_from_points(sampled_points, H, W)

    loss_map = F.mse_loss(predicted_map[i], pseudo_density)
else:
    # Inicjalizacja z BASNet (saliency) lub unsupervised DINO attention
    pseudo_map = dino_attention_map(batch["image"][i])
    historical_map_bank[img_id] = pseudo_map
    loss_map = F.mse_loss(predicted_map[i], pseudo_map)

# 3. Contrastive spatial loss
# Threshold predicted map → crowded vs background
threshold = predicted_map.mean(dim=(1, 2), keepdim=True)
crowded_mask = (predicted_map > threshold).float()
background_mask = 1.0 - crowded_mask

# Ekstrahuj cechy z crowded i background regionów
crowded_features = patch_features * crowded_mask.unsqueeze(-1)
background_features = patch_features * background_mask.unsqueeze(-1)

# Contrastive loss
loss_contrastive = info_nce_loss(crowded_features, background_features, temperature=0)

# 4. Count loss (integral = wiek)
predicted_count = predicted_map.sum(dim=(1, 2))
loss_count = F.mse_loss(predicted_count, age_labels.float())

loss = loss_coral + lambda_map * loss_map + lambda_contrast * loss_contrastive + lambda_count * loss_count

```

Parametry

- alpha (EMA): 0.7
- lambda_map: 1.0
- lambda_contrast: 1.0
- lambda_count: 1.0
- Temperatura kontrastyczna: $\tau=0.07$
- Inicjalizacja banku: BASNet (unsupervised saliency) lub DINO self-attention maps

Kluczowa różnica

Model sam generuje swoje pseudo-etykiety i iteracyjnie je ulepsza. Kontrastywny regularizator zapobiega degeneracji mapy do jednorodnego rozkładu.

Wariant 5: FOUND — DINO attention as pseudo-masks, self-trained 1×1 conv

Literatura

- Simeoni et al., "Unsupervised Object Localization: Observing the Background To Discover Objects (FOUND)", CVPR 2023
- URL: https://openaccess.thecvf.com/content/CVPR2023/papers/Simeoni_Unsupervised_Object_Localization_Observing_the_Background_To_Discover_Objects_CVPR_2023_paper.pdf

Opis

Ekstremalnie lekkie podejście: pojedyncza konwolucja 1×1 (770 parametrów) na zamrożonych cechach DINO. Najpierw odkrywa tło z cech DINO (patch o najmniejszym połączeniu z resztą), potem traktuje dopełnienie jako pierwszy plan (obiekty/przyrosty). Douczanie własnymi predykcjami.

Architektura

Backbone: DINO ViT-S/8 (zamrożony) – użyj ViT-S/8 dla drobniejszych patchy
Patch features: [B, N_patches, D] (D=384 dla ViT-S)
Self-attention maps: [B, N_heads, N_patches, N_patches] z ostatniej warstwy

Density head:

Conv2d(D, 1, kernel_size=1) → Sigmoid

Tylko 770 parametrów uczonych (D + 1 = 385, ale z bias: 385*2=770)

Input: patch_features reshaped → [B, D, H_patches, W_patches]

Output: [B, 1, H_patches, W_patches] → sigmoid → [B, H_patches, W_patches]

Procedura treningowa

FAZA 1: Generowanie pseudo-mask tła z cech DINO (bez treningu)

```
def discover_background(patch_features, attention_maps):
```

```
    """
```

```
    Odkrywa tło używając cech DINO.
```

```
    """
```

```
    # 1. Wybierz seed patch tła – patch o najmniejszej średniej uwagi
```

```
    attention_scores = attention_maps.mean(dim=1) # [B, N_patches, N_patches]
```

```
    # Sparsity-weighted attention (reweighting by sparsity)
```

```
    sparsity = (attention_scores > attention_scores.mean()).float().mean(dim=1) # [B]
```

```
    background_seed = sparsity.argmin(dim=1) # [B] – index seed patch
```

```
    # 2. Znajdź patche podobne do seed (cosine similarity)
```

```

    seed_features = patch_features[torch.arange(B), background_seed] # [B, D]
    similarities = F.cosine_similarity(
        patch_features, seed_features.unsqueeze(1), dim=-1
    ) # [B, N_patches]

    # 3. Threshold: patche o podobieństwie >  $\tau$  są tłem
    tau = 0.3
    background_mask = (similarities > tau).float() # [B, N_patches]

    # 4. Foreground = dopełnienie tła
    foreground_mask = 1.0 - background_mask # [B, N_patches]

    return foreground_mask, background_mask

# FAZA 2: Trenowanie głowicy 1x1 na pseudo-maskach

# Inicjalizacja
density_head = nn.Conv2d(D, 1, kernel_size=1)
nn.init.zeros_(density_head.bias)
nn.init.normal_(density_head.weight, std=0.01)

# Coarse background masks (z DINO attention)
coarse_fg, coarse_bg = discover_background(patch_features, attention_maps)

# Bilateral solver refinement (opcjonalny)
refined_fg = bilateral_solver(coarse_fg, images)

# Loss
predicted_fg = torch.sigmoid(density_head(patch_features_2d)) # [B, 1, H, W]
predicted_fg = predicted_fg.squeeze(1)

# Loss na maski tła (przez pierwsze m=100 iteracji)
loss_bg = F.binary_cross_entropy(predicted_fg, refined_bg)

# Self-prediction loss (po m=100 iteracjach)
# Model dopasowuje się do własnych binarized, refined predykcji
refined_prediction = bilateral_solver(predicted_fg.detach(), images)
binary_prediction = (refined_prediction > 0.5).float()
# Tylko gdy IoU > 0.5
iou = compute_iou(predicted_fg, binary_prediction)
self_mask = (iou > 0.5).float()
loss_self = (F.binary_cross_entropy(predicted_fg, binary_prediction, reduction='none'

loss = lambda_bg * loss_bg + lambda_self * loss_self

# FAZA 3: Połączenie z CORAL
# Density head wskazuje lokalizację, CORAL podaje wiek
loss_total = loss_coral + lambda_density * (loss_bg + loss_self)

```

Parametry

- tau (similarity threshold): 0.3
- mu (attention threshold): per-image mean
- lambda_bg: 1.5 (przez pierwsze 100 iteracji)
- lambda_self: 1.0 (po 100 iteracjach)
- m (switch to self-loss): 100 iteracji
- IoU threshold dla self-loss: 0.5
- Bilateral solver: opcjonalny, poprawia granice
- Trening: 500 iteracji, batch_size=50, ~2 epoki

Kluczowa różnica

Ekstremalnie lekka głowica (770 parametrów). Brak jakichkolwiek etykiet — model uczy się całkowicie z cech DINO. Pseudo-maski są generowane z self-attention DINO. To jest podejście w pełni nienadzorowane dla lokalizacji.

Uwaga dla otolitów

Zamiast odkrywać "obiekt na pierwszym planie", można zaadaptować FOUND do odkrywania struktur periodycznych (przyrostów) poprzez:

- Analizę spektralną cech DINO (FFT na mapie uwagi)
- Klastrowanie patchy na "przyrost" vs "tło"
- Zastosowanie FOUND do odkrywania pierwszego planu (otolit) na obrazie z tłem

Wariant 6: Additive MIL — Attention-based Multiple Instance Learning

Literatura

- Javed et al., "Additive MIL: Intrinsically Interpretable Multiple Instance Learning for Pathology", 2022
- URL: <https://arxiv.org/html/2206.01794>

Opis

Otolit traktowany jako "bag" patchy, wiek jako etykieta bag-level. Model uczy się wag uwagi dla każdego patcha, a addytywna dekompozycja pozwala na dokładną mapę cieplną wskazującą, które patche są istotne dla predykcji wieku.

Architektura

Backbone: DINO ViT-S/16 (zamrożony)

Featurizer: $f = \text{DINO}$ (zamrożony), output: $[B, N_{\text{patches}}, D]$

Attention MIL module:

Attention MLP: $\psi_m = \text{Linear}(D, 128) \rightarrow \text{Tanh} \rightarrow \text{Linear}(128, 1)$

Output: attention weights $\alpha_i = \text{softmax}(\psi_m(f(x_i))) \rightarrow [B, N_{\text{patches}}]$

Attended patch features: $m_i = \alpha_i * f(x_i) \rightarrow [B, N_{\text{patches}}, D]$

Predictor head (addytywna dekompozycja):

$\psi_p = \text{Linear}(D, K)$ ($K = \text{liczba klas wieku}$)

Patch contributions: $c_i = \psi_p(m_i) \rightarrow [B, N_{\text{patches}}, K]$

Slide/bag-level prediction: $p = \sum c_i \rightarrow [B, K]$

Heatmap per class: $\text{sigmoid}(c_i) \rightarrow [B, N_{\text{patches}}, K]$

excitatory: 0.5-1.0 (dodatni wkład)

inhibitory: 0.0-0.5 (ujemny wkład)

CORAL head (osobna, na [CLS] token lub na bag-level prediction):

$\text{Linear}(D, K-1)$ – standardowa głowica CORAL

Loss function

1. Bag-level loss (wiek otolitu)

`bag_prediction = patch_contributions.sum(dim=1) # [B, K]`

Cross-entropy na bag-level

`loss_bag = F.cross_entropy(bag_prediction, age_labels)`

2. CORAL loss (na CLS token lub bag prediction)

`coral_output = model.coral_head(cls_token) # [B, K-1]`

`loss_coral = coral_loss(coral_output, age_labels)`

3. Density loss – integral heatmap = wiek

Użyj heatmapy dla klasy odpowiadającej wiekowi

`density_map = torch.sigmoid(patch_contributions[:, :, age_labels]) # [B, N_patches]`

`density_map = density_map.view(B, H_patches, W_patches)`

`predicted_count = density_map.sum(dim=(1, 2))`

`loss_density = F.mse_loss(predicted_count, age_labels.float())`

4. Sparsity regularizer – encouraging focused attention

`attention_entropy = -(attention_weights * torch.log(attention_weights + 1e-8)).sum(dim=1)`

`loss_sparsity = attention_entropy # minimalizuj entropię`

`loss = loss_coral + lambda_bag * loss_bag + lambda_density * loss_density + lambda_sparsity * loss_sparsity`

Parametry

- `lambda_bag`: 1.0
- `lambda_density`: 0.5
- `lambda_sparsity`: 0.01
- Bag size: 48-1600 patchy (eksperymentuj)
- Hidden dim attention MLP: 128
- Learning rate: 1e-4

Kluczowa różnica

Model otrzymuje per-patch wkład w predykcję — addytywna dekompozycja daje dokładne, interpretowalne mapy cieplne. To podejście z histopatologii, gdzie slajd (bag) ma etykietę na poziomie obrazu, a model uczy się wskazywać regiony rakowe. Analogia: otolit (bag) ma wiek (etykieta), model uczy się wskazywać przyrosty.

Wariant 7: TokenCut — Graph-based foreground discovery from DINO features

Literatura

- Wang et al., "Self-Supervised Transformers for Unsupervised Object Discovery using Normalized Cut (TokenCut)", CVPR 2022
- URL: <https://inria.hal.science/hal-03585410/file/TokenCut.pdf>
- ArXiv: <https://arxiv.org/html/2209.00383v3>

Opis

Cechy patchy z DINO tworzą węzły grafu pełnego. Krawędzie ważone podobieństwem cosinusowym. Rozwiązanie Normalized Cut (drugie najmniejsza wartość własna) dzieli obraz na pierwszy plan (obiekty) i tło. Dla otolitów: podział na "przyrost" vs "tło między przyrostami".

Architektura

Backbone: DINO ViT-S/16 (zamrożony)

Patch features (keys): [B, N_patches, D] z ostatniej warstwy transformera

Graph construction:

- Węzły: N_patches patchy
- Krawędzie: pełny graf, waga = cosine similarity patch features
- Threshold: waga = 1 jeśli similarity $\geq \tau$, else $\epsilon=1e-5$
- $\tau = 0.2$

Normalized Cut:

- Macierz sąsiedztwa E: [N_patches, N_patches]
- Stopień D: diagonalna sumy wierszy E
- Rozwiązanie: $(D - E)y = \lambda Dy$ — druga najmniejsza wartość własna

- Podział na progu średniej eigenvectora
- Foreground: partition zawierająca max absolute value eigenvectora

Density head (opcjonalna, nałożona na wynik TokenCut):

Conv2d(D, 1, kernel_size=1) → ReLU

Douczone na pseudo-maskach z TokenCut

Implementacja

```
import torch
import torch.nn.functional as F
from scipy.linalg import eigh

def tokencut_density(patch_features, tau=0.2):
    """
    TokenCut: Normalized Cut na cechach DINO.
    Input: patch_features [B, N_patches, D]
    Output: foreground_mask [B, N_patches], density_map [B, N_patches]
    """
    B, N, D = patch_features.shape
    density_maps = []

    for b in range(B):
        features = patch_features[b] # [N, D]

        # 1. Macierz podobieństwa
        sim_matrix = F.cosine_similarity(
            features.unsqueeze(1), features.unsqueeze(0), dim=-1
        ) # [N, N]

        # 2. Thresholding
        adj_matrix = torch.where(sim_matrix >= tau,
                                torch.ones_like(sim_matrix),
                                torch.full_like(sim_matrix, 1e-5))

        # 3. Normalized Cut – druga najmniejsza wartość własna
        D = torch.diag(adj_matrix.sum(dim=1))
        L = D - adj_matrix # Laplacian
        #  $D^{-1} L y = \lambda y \rightarrow$  generalized eigenvalue problem
        D_inv = torch.diag(1.0 / (adj_matrix.sum(dim=1) + 1e-8))
        L_normalized = D_inv @ L

        # Rozwiąż (użyj scipy dla stabilności numerycznej)
        eigenvalues, eigenvectors = eigh(L_normalized.cpu().numpy())
        # Druga najmniejsza wartość własna (pierwsza = 0, trywialna)
        fiedler_vector = torch.tensor(eigenvectors[:, 1], device=patch_features.device)

        # 4. Podział na progu średniej
        threshold = fiedler_vector.mean()
        foreground = (fiedler_vector > threshold).float()

        # 5. Density map – użyj foreground jako pseudo-density
        # Normalizuj przez liczbę patchy w foreground
        density = foreground / (foreground.sum() + 1e-8)
        density_maps.append(density)
```

```

        return torch.stack(density_maps) # [B, N_patches]

# W treningu:
# 1. Oblicz pseudo-density z TokenCut (bez gradientu – zamrożone)
pseudo_density = tokencut_density(patch_features.detach()) # [B, N_patches]

# 2. Density head uczy się na pseudo-density
predicted_density = model.density_head(patch_features) # [B, N_patches]
predicted_density = F.relu(predicted_density)

# 3. Loss
loss_tokencut = F.mse_loss(predicted_density, pseudo_density)

# 4. Integral constraint
predicted_count = predicted_density.sum(dim=1)
loss_count = F.mse_loss(predicted_count, age_labels.float())

# 5. CORAL loss
loss_coral = coral_loss(coral_output, age_labels)

loss = loss_coral + lambda_tokencut * loss_tokencut + lambda_count * loss_count

```

Parametry

- tau (similarity threshold): 0.2 (dla DINO ViT-S/16)
- lambda_tokencut: 1.0
- lambda_count: 0.5
- TokenCut wykonany bez gradientu (jako generator pseudo-etykiet)
- Density head: Conv2d(D, 1, 1) → ReLU

Kluczowa różnica

Brak uczenia dla samego podziału — TokenCut to algorytm grafowy działający na cechach DINO. Density head jest douczana na pseudo-maskach z TokenCut. To podejście nie wymaga żadnych etykiet lokalizacji — tylko cechy DINO i algebra liniowa.

Uwaga dla otolitów

TokenCut odkrywa "główny obiekt" — dla otolitów może to oznaczać odkrycie całego otolitu na tle obrazu, nie pojedynczych przyrostów. Aby odkryć przyrosty, można:

- Zastosować TokenCut rekursywnie (najpierw otolit, potem przyrosty wewnątrz)
- Zastosować analizę spektralną na macierzy podobieństwa (znajdowanie struktur periodycznych)
- Użyć TokenCut na cechach z różnych warstw DINO (różne skale)

Wariant 8: Semi-supervised — Mały zbiór z ręcznymi etykietami przyrostów + duży zbiór z wiekiem

Literatura

- Liu et al. (2020). "Towards Using Count-level Weak Supervision for Crowd Counting." arXiv:2003.00164. <https://arxiv.org/pdf/2003.00164v1>
- Cholakal et al. (2019). "Object Counting and Instance Segmentation with Image-Level Supervision." CVPR 2019. https://openaccess.thecvf.com/content_CVPR_2019/papers/Cholakal_Object_Counting_and_Instance_Segmentation_With_Image-Level_Supervision_CVPR_2019_paper.pdf
- Bojesen et al. (2024). "Annotating otoliths with a deep generative model." ICES JMS. <https://academic.oup.com/icesjms/article/81/1/55/7341839>
- Cayetano et al. (2024). "Fish age reading using deep learning methods for object-detection and segmentation." ICES JMS. <https://academic.oup.com/icesjms/advance-article/doi/10.1093/icesjms/fsae020/7614790>
- Bahat & Shakhnarovich (2020). "Density-aware Counting." arXiv:2003.00164 (MATT — semi-supervised setup: small fully-labeled + large count-only)
- Ravaev et al. (2021). "Semi-supervised Density Estimation for Crowd Counting." (podejście semi-supervised z pseudo-etykietami propagowanymi z małego zbioru)

Opis

W tym wariancie zakładamy dwa zbiory danych:

1. **Zbiór A (mały, np. 20-50 obrazów):** Otolity z ręcznie oznaczonymi przyrostami — etykiety w formacie punktowym (współrzędne x,y środka każdego przyrostu) lub maski binarne (piksele przyrostów). Liczba etykiet = wiek otolitu.
2. **Zbiór B (duży, np. 1000+ obrazów):** Otolity tylko z etykietą wieku (liczba całkowita) — bez żadnych oznaczeń lokalizacji przyrostów.

Głowica density jest trenowana w trybie hybrydowym:

- Na zbiorze A: bezpośredni nadzór punktowy/maskowy — model uczy się faktycznego rozmieszczenia przyrostów
- Na zbiorze B: słaby nadzór — integral mapy density = wiek (count-level supervision)

To podejście jest bezpośrednio inspirowane pracą MATT (Liu et al., 2020), która używa małego zbioru z adnotacjami punktowymi (location-level) i dużego zbioru z samą liczbą (count-level). Autorzy pokazali, że zaledwie 5-10% obrazów z pełnymi adnotacjami + 90-95% z samą liczbą daje wyniki zbliżone do pełnego nadzoru punktowego.

Format danych

```
# Zbiór A – ręczne etykiety punktowe przyrostów
{
  "image_path": "otolith_001.png",
  "age": 5,
  "increment_points": [
    [x1, y1], # współrzędne środka przyrostu 1
    [x2, y2], # współrzędne środka przyrostu 2
    ...
  ]
}

# Alternatywnie: maska binarna
{
  "image_path": "otolith_001.png",
  "age": 5,
  "increment_mask": "otolith_001_mask.png" # binarna maska pikseli przyrostów
}

# Zbiór B – tylko wiek
{
  "image_path": "otolith_100.png",
  "age": 7
  # brak increment_points lub increment_mask
}
```

Architektura

Backbone: DINO ViT-S/16 (zamrożony)
Patch features: [B, N_patches, D] (D=384)

CORAL head:

Input: [B, D] (token [CLS])
Output: [B, K-1] (K = max_age + 1)
Loss: CORAL ordinal loss

Density head (główna):

Conv2d(D, 256, 1) → ReLU → Conv2d(256, 128, 1) → ReLU → Conv2d(128, 1, 1) → ReLU
Input: [B, D, H_patches, W_patches]
Output: [B, 1, H_patches, W_patches] → ReLU (nieujemna)

Density head (pomocnicze – opcjonalne, jak w MATT):

4× Conv2d(D, 128, 1) → ReLU → Conv2d(128, 64, 1) → ReLU → Conv2d(64, 1, 1) → ReLU
(porzucone po treningu, consistency loss z różnymi jądrami Gaussa)

Generowanie ground-truth density maps z etykiet punktowych (zbiór A)

```
def make_density_map_from_points(points, H_patches, W_patches, sigma=1.0):
    """
    Tworzy mapę gęstości z punktów przyrostów.
    Każdy punkt zamieniany na Gaussa – suma daje mapę gęstości.
    To standardowa metoda z crowd counting.

    Args:
        points: list of [x, y] w współrzędnych patchy
        H_patches, W_patches: rozmiar mapy w patchach
        sigma: odchylenie Gaussa (w patchach)
    Returns:
        density_map: [H_patches, W_patches] – nieujemna mapa gęstości
    """
    density_map = torch.zeros(H_patches, W_patches)

    for x, y in points:
        # Twórz Gaussa wokół punktu
        for dy in range(-3, 4): # okno 7x7
            for dx in range(-3, 4):
                py, px = int(y) + dy, int(x) + dx
                if 0 <= py < H_patches and 0 <= px < W_patches:
                    density_map[py, px] +=
                        math.exp(-(dx**2 + dy**2) / (2 * sigma**2))

    return density_map

def make_density_map_from_mask(mask, H_patches, W_patches):
    """
    Alternatywnie: maska binarna → mapa gęstości.
    Każdy piksel maski = 1, reszta = 0.
    Opcjonalnie: wygładzenie filtrem Gaussa.
    """
    # Resize maski do rozdzielczości patchy
    mask_resized = F.interpolate(
        mask.unsqueeze(0).unsqueeze(0).float(),
        size=(H_patches, W_patches),
        mode='bilinear',
        align_corners=False
    ).squeeze()
    # Opcjonalne wygładzenie
    mask_smoothed = gaussian_blur(mask_resized, kernel_size=3, sigma=1.0)
    return mask_smoothed
```

Loss function

```
# =====
# Zbiór A – pełny nadzór (punktowy/maskowy)
# =====

for batch_a in dataloader_annotated:
    images_a = batch_a["image"]          # [B_a, 3, H, W]
    points_a = batch_a["points"]         # list of [N_i, 2] – współrzędne przyrostów
```

```

age_a = batch_a["age"] # [B_a]

# Forward pass
patch_features_a = backbone(images_a) # [B_a, N_patches, D]
cls_token_a = patch_features_a[:, 0] # [B_a, D]

coral_output_a = coral_head(cls_token_a) # [B_a, K-1]
density_map_a = density_head(
    patch_features_a.permute(0, 2, 1).view(B_a, D, H_patches, W_patches)
).squeeze(1) # [B_a, H_patches, W_patches]

# 1. CORAL loss
loss_coral_a = coral_loss(coral_output_a, age_a)

# 2. Ground-truth density maps z punktów
gt_density_a = torch.stack([
    make_density_map_from_points(pts, H_patches, W_patches, sigma=1.0)
    for pts in points_a
]) # [B_a, H_patches, W_patches]

# 3. Density MSE loss – bezpośredni nadzór!
loss_density_supervised = F.mse_loss(density_map_a, gt_density_a)

# 4. Count consistency – integral = wiek
predicted_count_a = density_map_a.sum(dim=(1, 2))
loss_count_a = F.mse_loss(predicted_count_a, age_a.float())

loss_a = loss_coral_a + lambda_supervised * loss_density_supervised + lambda_count

# =====
# Zbiór B – słaby nadzór (count-level)
# =====

for batch_b in dataloader_weak:
    images_b = batch_b["image"] # [B_b, 3, H, W]
    age_b = batch_b["age"] # [B_b]

    # Forward pass
    patch_features_b = backbone(images_b) # [B_b, N_patches, D]
    cls_token_b = patch_features_b[:, 0]

    coral_output_b = coral_head(cls_token_b)
    density_map_b = density_head(
        patch_features_b.permute(0, 2, 1).view(B_b, D, H_patches, W_patches)
    ).squeeze(1) # [B_b, H_patches, W_patches]

    # 1. CORAL loss
    loss_coral_b = coral_loss(coral_output_b, age_b)

    # 2. Count loss – integral = wiek (słaby nadzór)
    predicted_count_b = density_map_b.sum(dim=(1, 2))
    loss_count_b = F.mse_loss(predicted_count_b, age_b.float())

    # 3. Opcjonalnie: auxiliary consistency loss (jak w MATT, Wariant 3)
    # Pomocnicze głowice z różnymi jądrami Gaussa
    if use_aux_heads:

```

```

    for i, aux_head in enumerate(aux_heads):
        aux_density = aux_head(
            patch_features_b.permute(0, 2, 1).view(B_b, D, H_patches, W_patches)
        ).squeeze(1)
        aux_count = aux_density.sum(dim=(1, 2))
        loss_aux_count = F.mse_loss(aux_count, age_b.float())

        # Consistency: skonwoluwowana aux powinna pasować do głównej (detached)
        kernel = gaussian_kernels[i]
        aux_smoothed = F.conv2d(
            aux_density.unsqueeze(1), kernel, padding=kernel_size // 2
        ).squeeze(1)
        loss_consistency = F.mse_loss(aux_smoothed, density_map_b.detach())

        loss_aux = loss_aux_count + loss_consistency

    loss_b = loss_coral_b + lambda_count_b * loss_count_b + lambda_aux_b * sum(loss_aux)
else:
    loss_b = loss_coral_b + lambda_count_b * loss_count_b

# =====
# Łączny loss na epokę
# =====

# Pętla treningowa: naprzemiennie batche z A i B, lub mieszane batche
# Opcja 1: naprzemiennie – jeden batch z A, jeden z B
# Opcja 2: mieszane – losowanie proporcji (np. 1:4 = 20% A, 80% B)

loss_total = loss_a + loss_b
loss_total.backward()
optimizer.step()

```

Strategie treningowe

Strategia A: Naprzemiennie (alternating)

```

# Epoka: połowa batchy z A, połowa z B
for epoch in range(epochs):
    # Batch z A
    for batch_a in dataloader_annotated:
        loss = compute_loss_a(batch_a)
        loss.backward()
        optimizer.step()

    # Batch z B
    for batch_b in dataloader_weak:
        loss = compute_loss_b(batch_b)
        loss.backward()
        optimizer.step()

```

Strategia B: Mieszane (mixed batching)

```
# Każdy batch: mix obrazów z A i B
for epoch in range(epochs):
    for batch in mixed_dataloader: # 20% z A, 80% z B
        mask_annotated = batch["has_annotations"] # [B] bool

        # Density loss tylko dla obrazów z A
        if mask_annotated.any():
            loss_density_supervised = compute_supervised_loss(batch[mask_annotated])
        else:
            loss_density_supervised = 0.0

        # Count loss dla wszystkich
        loss_count = compute_count_loss(batch)
        loss_coral = compute_coral_loss(batch)

        loss = loss_coral + lambda_supervised * loss_density_supervised + lambda_count * loss_count
        loss.backward()
        optimizer.step()
```

Strategia C: Pre-train na A, finetune na A+B

```
# Faza 1: Pre-train głowicy density na małym zbiorze A (50-100 epok)
for epoch in range(pretrain_epochs):
    for batch_a in dataloader_annotated:
        loss = compute_loss_a(batch_a) # tylko supervised density + CORAL
        loss.backward()
        optimizer.step()

# Faza 2: Finetune na A+B (głównie count-level + zachowanie supervised)
for epoch in range(finetime_epochs):
    for batch in mixed_dataloader:
        loss = compute_loss_mixed(batch)
        loss.backward()
        optimizer.step()
```

Strategia D: Pseudo-label propagation (self-training)

```
# Faza 1: Pre-train na A (supervised)
# Faza 2: Generuj pseudo-density maps dla B używając nauczonego modelu
# Faza 3: Retrain na A + B (B z pseudo-density jako soft targets)

for epoch in range(pseudo_epochs):
    # Generuj pseudo-etykiety dla B
    model.eval()
    pseudo_density_maps = {}
    for batch_b in dataloader_weak:
        with torch.no_grad():
            density = model.density_head(batch_b)
            pseudo_density_maps[batch_b["id"]] = density
```

```

# Trenuj na A (hard labels) + B (pseudo labels, z confidence weighting)
model.train()
for batch in mixed_dataloader:
    if batch["has_annotations"]:
        # Hard target
        gt_density = make_density_from_points(batch["points"])
        loss_density = F.mse_loss(predicted_density, gt_density)
    else:
        # Soft pseudo-target z confidence
        pseudo = pseudo_density_maps[batch["id"]]
        confidence = pseudo.max(dim=(1,2)).values # [B]
        loss_density = F.mse_loss(predicted_density, pseudo) * confidence.mean()

    loss = loss_coral + lambda_density * loss_density
    loss.backward()
    optimizer.step()

```

Parametry

Parametr	Wartość domyślna	Zakres do strojenia
n_annotated (liczba obrazów z etykietami)	30	[10, 20, 30, 50, 100]
lambda_supervised	1.0	[0.5, 1.0, 2.0, 5.0]
lambda_count_a	0.5	[0.1, 0.5, 1.0]
lambda_count_b	1.0	[0.5, 1.0, 2.0]
lambda_aux_b (jeśli używamy aux heads)	1.0	[0.5, 1.0, 2.0]
sigma (Gaussa dla density map z punktów)	1.0	[0.5, 1.0, 1.5, 2.0]
mix_ratio (proporcja A:B w batchu mieszanym)	0.2	[0.1, 0.2, 0.3, 0.5]
pretrain_epochs (Strategia C/D)	50	[20, 50, 100]
finetune_epochs (Strategia C/D)	100	[50, 100, 200]
use_aux_heads	true	[true, false]
pseudo_confidence_threshold (Strategia D)	0.5	[0.3, 0.5, 0.7]
gaussian_kernels (jeśli aux heads)	[3, 3], [5, 5], [3, 5], [5, 3]	—

Kluczowa różnica względem innych wariantów

- **Wariant 1 (Baseline):** tylko count-level, brak jakiejkolwiek wiedzy o lokalizacji
- **Wariant 5 (FOUND):** pseudo-maski z DINO attention, ale nie używa żadnych ręcznych etykiet
- **Wariant 8 (ten):** bezpośredni nadzór na małym zbiorze + count-level na dużym — model uczy się prawdziwego rozmieszczenia przyrostów z kilku przykładów i generalizuje na resztę

To podejście powinno dać najlepsze wyniki lokalizacji ze wszystkich wariantów, ponieważ:

1. Mały zbiór z etykietami dostarcza prawdziwego sygnału o tym, gdzie są przyrosty
2. Duży zbiór z samym wiekiem zapobiega przeuczeniu i wymusza integral = wiek
3. Pseudo-label propagation (Strategia D) pozwala modelowi samemu ulepszać swoje predykcje na dużym zbiorze

Oczekiwane wyniki

- **Age MAE:** powinno być porównywalne lub lepsze niż inne warianty (CORAL head dostaje dodatkowy sygnał gradientu z supervised density)
- **Lokalizacja przyrostów:** powinna być **znacznie lepsza** niż w wariantach czysto słabonadzorowanych – model widział prawdziwe przyrosty na małym zbiorze
- **Count error:** integral mapy density powinien dobrze odpowiadać wiekowi, ponieważ jest bezpośrednio nadzorowany na obu zbiorach
- **IoU/Precision/Recall:** jedyne warianty z mierzalną lokalizacją przyrostów (jeśli zbiór testowy A ma etykiety)

Eksperymenty z rozmiarem zbioru A

Kluczowy eksperyment: jak mały może być zbiór A, aby model nadal działał?

```
ablation_n_annotated:
- n_annotated: 5      # ekstremalnie mały
- n_annotated: 10
- n_annotated: 20
- n_annotated: 30
- n_annotated: 50
- n_annotated: 100   # referencja

# Dla każdego n: uruchom 3 strategie treningowe (A, C, D)
# Metryki: Age MAE, Count error, IoU (na zbiorze testowym z etykietami)
```

Walidacja lokalizacji (zbiór testowy z etykietami)

Jeśli dostępny jest osobny zbiór testowy z ręcznymi etykietami przyrostów (nie używany w treningu):

```
def evaluate_localization(model, test_loader_with_annotations):
    """
    Ewaluacja lokalizacji przyrostów na zbiorze testowym z etykietami.
    """
    all_iou = []
    all_precision = []
    all_recall = []
    all_f1 = []
    all_auprc = []

    for batch in test_loader_with_annotations:
        images = batch["image"]
```

```

gt_points = batch["points"] # list of [N_i, 2]
gt_masks = batch.get("masks", None) # opcjonalnie

# Predykcja
density_map = model.density_head(images) # [B, H, W]

for i in range(len(images)):
    # Threshold mapy density → binarna maska przyrostów
    threshold = density_map[i].quantile(0.9) # top 10% jako przyrosty
    predicted_mask = (density_map[i] > threshold).float()

    # GT maska z punktów
    if gt_masks is not None:
        gt_mask = gt_masks[i]
    else:
        gt_mask = points_to_mask(gt_points[i], H_patches, W_patches, radius=radius)

    # Metryki
    iou = compute_iou(predicted_mask, gt_mask)
    precision, recall = compute_precision_recall(predicted_mask, gt_mask)
    f1 = 2 * precision * recall / (precision + recall + 1e-8)

    all_iou.append(iou)
    all_precision.append(precision)
    all_recall.append(recall)
    all_f1.append(f1)

# AUPRC
auprc = compute_auprc(density_map, gt_mask)

return {
    "IoU": np.mean(all_iou),
    "Precision": np.mean(all_precision),
    "Recall": np.mean(all_recall),
    "F1": np.mean(all_f1),
    "AUPRC": auprc,
}

```

Metryki ewaluacji (wspólne dla wszystkich wariantów)

Metryki wieku (głowica CORAL)

- **Age MAE:** średni błąd bezwzględny wieku (główne kryterium)
- **Age RMSE:** pierwiastek z błędu średniokwadratowego
- **Ordinal accuracy:** % poprawnie przewidzianych wieków
- **Bias:** średnia różnica (predicted - true) — detekcja systematycznego odchylenia
- **CV (coefficient of variation):** std / mean — miara powtarzalności

Metryki density (głowica density)

- **Count error:** $|\sum \text{density_map} - \text{true_age}| / \text{true_age}$ — błąd sumy mapy
- **Sparsity:** ułamek patchy z wartością $> \text{threshold}$ ($\text{np.} > 0.1 * \text{max}$)
- **Entropy:** $-\sum p_i * \log(p_i)$ na znormalizowanej mapie — umiarkowana entropia = dobra lokalizacja
- **Radial consistency:** korelacja mapy gęstości z odległością od centrum otolitu (przyrosty powinny być rozmieszczone radialnie)
- **Concentric ring alignment:** miara zgodności mapy z wzorcem koncentrycznym (opcjonalne — jeśli otolit ma strukturę koncentryczną)

Wizualizacje

- **Overlay density map:** mapa gęstości nałożona na obraz otolitu (heatmap)
- **Attention map DINO:** mapa uwagi DINO (CLS token attention) — dla porównania
- **Per-age visualization:** osobne wizualizacje dla różnych klas wieku
- **Radial profile:** profil gęstości wzdłuż osi radialnej otolitu

Opcjonalna walidacja lokalizacji (jeśli dostępny mały zbiór z ręcznymi etykietami)

- **IoU:** Intersection over Union między mapą density a ręcznymi maskami przyrostów
- **Precision/Recall/F1:** dla detekcji przyrostów (po thresholding mapy density)
- **AUPRC:** Area Under Precision-Recall Curve

Plan eksperymentów

Kolejność uruchamiania

1. **Wariant 1 (Baseline)** — punkt odniesienia, najprostszy
2. **Wariant 5 (FOUND)** — w pełni nienadzorowany, sprawdza czy DINO attention samo wskazuje struktury
3. **Wariant 2 (Cholakkal)** — dodaje głowicę klasyfikacji generującą pseudo-lokalizacje
4. **Wariant 3 (MATT)** — dodaje głowice pomocnicze z consistency loss
5. **Wariant 4 (Count2Density)** — Historical Map Bank + contrastive
6. **Wariant 6 (Additive MIL)** — attention MIL z histopatologii
7. **Wariant 7 (TokenCut)** — grafowe odkrywanie z DINO
8. **Wariant 8 (Semi-supervised)** — mały zbiór z ręcznymi etykietami + duży z wiekiem; uruchom jako ostatni — wymaga przygotowania ręcznych etykiet i jest punktem odniesienia dla lokalizacji przyrostów

Konfiguracja eksperymentów

```
# Wspólne ustawienia
backbone: dinov2_vits14 # lub dino_vits16 – sprawdzić w istniejącym projekcie
frozen: true
input_size: 224 # lub z projektu
batch_size: 32
optimizer: adam
lr_head: 1e-4
lr_backbone: 0 # zamrożony
epochs: 100
early_stopping_patience: 15

# Variants
variants:
  - name: "v1_baseline"
    config:
      lambda_density: [0.1, 0.5, 1.0, 2.0]

  - name: "v2_cholakkal"
    config:
      lambda_class: 1.0
      lambda_density: 1.0
      lambda_rank: 0.1
      subitizing_range: [1, 4]

  - name: "v3_matt"
    config:
      n_aux_heads: 4
      lambda_primary: 1.0
      lambda_aux: 1.0
      gaussian_kernels: [[3,3], [5,5], [3,5], [5,3]]

  - name: "v4_count2density"
    config:
      alpha_ema: 0.7
      lambda_map: 1.0
      lambda_contrast: 1.0
      temperature: 0.07
      init: "dino_attention" # lub "basnet"

  - name: "v5_found"
    config:
      tau: 0.3
      lambda_bg: 1.5
      lambda_self: 1.0
      switch_iteration: 100
      iou_threshold: 0.5
      max_iters: 500

  - name: "v6_additive_mil"
    config:
      lambda_bag: 1.0
      lambda_density: 0.5
      lambda_sparsity: 0.01
      attention_hidden: 128
```

```

    bag_size: [48, 256, 1024]

- name: "v7_tokencut"
  config:
    tau: 0.2
    lambda_tokencut: 1.0
    lambda_count: 0.5

- name: "v8_semisupervised"
  config:
    n_annotated: [5, 10, 20, 30, 50, 100]
    lambda_supervised: [0.5, 1.0, 2.0, 5.0]
    lambda_count_a: 0.5
    lambda_count_b: 1.0
    sigma: [0.5, 1.0, 1.5, 2.0]
    mix_ratio: [0.1, 0.2, 0.3, 0.5]
    strategies: ["alternating", "mixed", "pretrain_finetune", "pseudo_label"]
    use_aux_heads: [true, false]
    pretrain_epochs: 50
    finetune_epochs: 100
    pseudo_confidence_threshold: [0.3, 0.5, 0.7]

```

Logowanie

- Zapisuj checkpoint'y najlepszego modelu (wg Age MAE)
- Loguj wszystkie metryki do TensorBoard / W&B
- Zapisuj wizualizacje overlay co N epok
- Zapisuj konfigurację eksperymentu jako JSON
- Tabela porównawcza wszystkich wariantów na końcu

Instrukcje dla Claude Code

1. Najpierw zbadaj istniejący projekt (Krok 0) i wypisz znalezione komponenty
2. Zaimplementuj każdy wariant jako osobny moduł — nie modyfikuj istniejącego kodu, dodawaj nowe pliki (np. `variants/v1_baseline.py`, `variants/v2_cholakkal.py`, itd.)
3. Współdziel backbone i CORAL head — nie dubluj kodu DINO/CORAL, importuj z istniejącego projektu
4. Każdy wariant musi mieć:
 - Konfigurację jako argumenty lub plik YAML
 - Funkcję `train_variant()` z własną pętlą treningową
 - Funkcję `evaluate_variant()` z wszystkimi metrykami
 - Zapis wizualizacji (overlay density map)
5. Użyj istniejącego datasetu i dataloader'a — nie twórz nowych
6. Zapisuj wyniki do tabeli porównawczej (CSV/JSON) na końcu wszystkich eksperymentów

7. Dla każdego wariantu uruchom krótki test (1-2 epoki) przed pełnym treningiem, aby zweryfikować, że kod działa
8. Zaimplementuj funkcję `run_all_experiments()`, która uruchamia wszystkie warianty sekwencyjnie i zapisuje wyniki

Bibliografia

1. Caron et al. (2021). "Emerging Properties in Self-Supervised Vision Transformers." arXiv:2104.14294. <https://arxiv.org/abs/2104.14294>
2. Wang et al. (2022). "Self-Supervised Transformers for Unsupervised Object Discovery using Normalized Cut (TokenCut)." CVPR 2022. <https://inria.hal.science/hal-03585410/file/TokenCut.pdf>
3. Simeoni et al. (2023). "Unsupervised Object Localization: Observing the Background To Discover Objects (FOUND)." CVPR 2023. https://openaccess.thecvf.com/content/CVPR2023/papers/Simeoni_Unsupervised_Object_Localization_Observing_the_Background_To_Discover_Objects_CVPR_2023_paper.pdf
4. Cholakal et al. (2019). "Object Counting and Instance Segmentation with Image-Level Supervision." CVPR 2019. https://openaccess.thecvf.com/content_CVPR_2019/papers/Cholakal_Object_Counting_and_Instance_Segmentation_With_Image-Level_Supervision_CVPR_2019_paper.pdf
5. Liu et al. (2020). "Towards Using Count-level Weak Supervision for Crowd Counting." arXiv:2003.00164. <https://arxiv.org/pdf/2003.00164v1>
6. "Count2Density: Crowd Density Estimation without Location-level Annotations." 2025. arXiv:2509.03170. <https://arxiv.org/pdf/2509.03170>
7. Javed et al. (2022). "Additive MIL: Intrinsically Interpretable Multiple Instance Learning for Pathology." arXiv:2206.01794. <https://arxiv.org/html/2206.01794>
8. Cao, Mirjalili, Raschka (2020). "Rank Consistent Ordinal Regression for Neural Networks with Application to Age Estimation." arXiv:1901.07884. <https://arxiv.org/abs/1901.07884>
9. coral-pytorch (GitHub). <https://github.com/Raschka-research-group/coral-pytorch>
10. Amir et al. "Deep ViT Features as Dense Visual Descriptors." <https://dino-vit-features.github.io/paper.pdf>
11. Moen et al. (2018). "Automatic interpretation of otoliths using deep learning." PLoS ONE. <https://pmc.ncbi.nlm.nih.gov/articles/PMC6296523/>
12. Bojesen et al. (2024). "Annotating otoliths with a deep generative model." ICES JMS. <https://academic.oup.com/icesjms/article/81/1/55/7341839>
13. Cayetano et al. (2024). "Fish age reading using deep learning methods for object-detection and segmentation." ICES JMS. <https://academic.oup.com/icesjms/advance-article/doi/10.1093/icesjms/fsae020/7614790>
14. Liu et al. (2020). "Towards Using Count-level Weak Supervision for Crowd Counting (MATT)." arXiv:2003.00164. <https://arxiv.org/pdf/2003.00164v1> — semi-supervised setup: small fully-labeled + large count-only set

